

Глубокое прокидывание опций

При выполнении задач разработчики часто создают контролы, которые принимают в себя опции, а потом распределяют их между своими детьми.

Иногда опций бывает слишком много или они пересекаются между детьми, поэтому для упрощения используется конструкция `scope="{ { _options } }"` (в react-компонентах для этого используется конструкция `{...props}`).

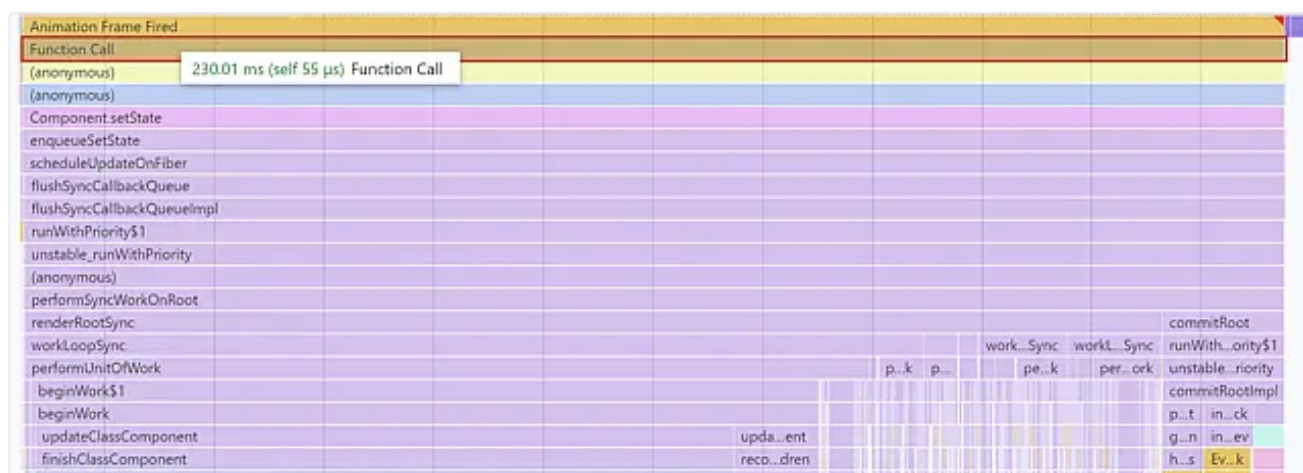
Это очень мощная конструкция и некоторые вещи без неё сделать невозможно (например, контролы высшего порядка), но у неё есть свои подводные камни. Самой большой проблемой является то, что так прокидываются все опции, даже те, которые не нужны, и это может приводить к проблемам с производительностью.

См. демо-пример

В этом примере есть два контрола, каждый из которых содержит в себе список и счётчик, и принимает две опции - значение счётчика и элементы списка. Также есть две кнопки, которые меняют значение счётчика в соответствующем списке. Левый контрол передаёт опции через `scope="{ { _options } }`", а правый передаёт каждую опцию по-отдельности.

При нажатии на кнопки будет заметно, что левый счётчик обновляется значительно медленнее. Давайте выясним, почему.

Для начала попробуем сделать это с помощью встроенного профилировщика Chrome:



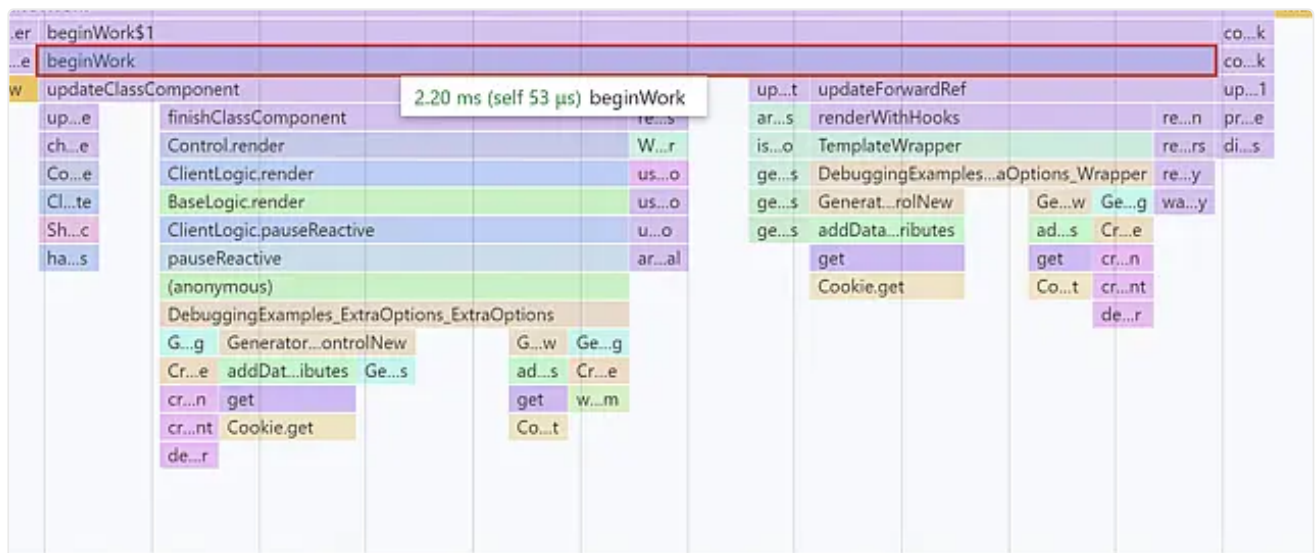
Профиль состоит из двух основных частей - вычисление нового состояния приложения (всё, что ниже `renderRootSync`) и применение этого нового состояния (всё, что ниже `commitRoot`).

На вторую часть, в большинстве случаев, можно повлиять только косвенно (например, уменьшить количество элементов на странице), поэтому следует сосредоточиться на первой части.



Если посмотреть левую часть подробнее, можно увидеть, что почти всё время занимает вычисление нового состояния списка. Но список не изменился, так что всё это время можно считать потерянным.

Теперь посмотрим профиль для правого списка:



Совершенно другая картина — на вычисление состояния теперь почти не уходит времени. И если посмотреть на него поближе, то можно увидеть, что список даже не участвовал в синхронизации (в профиле нет `DebuggingExamples_ExtraOptions_List`), отсюда и выигрыш в производительности.

Подведём итоги:

1. Использовать `score="{{ _options }}"` (или `{...props}`) нужно очень аккуратно, всегда лучше прокидывать опции по отдельности.
2. Нужно обращать внимание на лишние контролы, участвующие в синхронизациях. Особенно опасны те контролы, внутри которых есть списки (не важно платформенные или просто цикл `for`), т.к. это легко может замедлить синхронизацию в несколько раз.